CSE 144 Final Project Report

Transfer Learning Challenge

Evan Kaiden (CruzID: 2117192)
Rami Al Habash (CruzID: 2123611)

Spring 2026

1 Introduction

1. **Problem goal and setting.**

   Data: 100 classes with approximately 10 images per class, each class is part of a higher level semantic class, namely: Food, Flowers, Cars, and Planes. Each of the four groups has 25 classes for which consist of images of the main group type.

   Setting: This is a small data setting, $\sim$10 images per class and 100 classes poses a difficult optimization problem. However we can overcome this using transfer learning on pretrained models.

   Goal: We classify the test images with a model trained on the train set as best as we can.

2. **Why transfer learning is appropriate.**

   There are two main reasons:

   - It is a small data setting
   - The classes present in this represent natural images found in almost any large dataset used to train large networks. For example, ImageNet contains training samples of the exact classes present in this dataset. Because of this we know that the general knowledge of a model pretrained on such a dataset will generalize well to our setting.

3. **Brief summary of your approach and main result.**

   We load in a pretrained model from timm, and linearly probe the network for 20 epochs, we save the model with the highest val accuracy per fold. We generate the test results by running the original test images along with 5 augmented views of that image and averaging the prediction we then further average the predicted class probabilities over all folds models. We achieve a top-1 accuracy of 95.454%

on the public test data. model weights for one training fold can be found here: https://drive.google.com/file/d/1inQDS0OJAtrXXVqMCtXYL9DOrSH3xZho/view?usp=sharing we only provide 1 fold as all 10 folds would be 15GB which is not realistic to upload to Google Drive. If you would like the models for the other 9 folds let us know and we would be happy to provide them.

2 Dataset

1. **Number of classes and dataset sizes (train/val/test).**

   Number of classes: 100
   Train set size: ∼900 images
   Val set size: ∼100 images
   Test set size: 1036 images

2. **Directory structure and label mapping details (ensure labels 0–99 match folders).**

   We explicitly map each class to the proper folder as by default PyTorch maps in lexicographical order which has unwanted behavior in this case. We do this by making a class `SortedImageFolder` which inherits from `torchvision.datasets.ImageFolder` and we define a `find_classes(self, directory)` to map each image to its class with the proper **folder** $\rightarrow$ **cls** mapping.

3. **Preprocessing (resize, normalization) and data augmentation used.**

   We use the following transforms from `torchvision.transforms` on the trainset:

```
RandomResizedCrop(
    size=(378, 378),
    scale=(0.6, 1.0),
    interpolation=InterpolationMode.BICUBIC),
RandomHorizontalFlip(p=0.5),
RandAugment(num_ops=2, magnitude=9),
Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5]),
RandomErasing(p=0.25, scale=(0.02, 0.2), ratio=(0.3, 3.3),
   value='random'),
```

   For the validation set we only resize and normalize the images.

3 Implementation

3.1 Model

1. **Pretrained backbone used (e.g., ResNet/EfficientNet/ViT) and why.**

   `Vit_so400m_patch14_siglip_378.v2_webli`

A ViT trained on an extremely large dataset is best suited for this task as they generalize the best and have proved that they perform much better than CNN architectures when pretrained on a large enough dataset.

2. **Architecture changes (classifier head, pooling, dropout, etc.).**

   We remove the original classification head and replace with a un-trained classification head (single linear layer) to output logits for 100 classes.

3. **Fine-tuning strategy (frozen layers, unfreezing schedule).**

   All layers in the network except for the classification head. We had tested gradual unfreezing of the later layers in the network, but did not observe a gain in performance so this was scrapped to keep the code simple and clean.

3.2 Training

1. **Loss function and optimizer.**

   We use negative log likelihood loss and AdamW in the case of a routing network, for the normal network we use cross entropy loss and AdamW. We use NLL loss in case 1 because we are working with log probs.

   In both cases we set `label_smoothing = 0.1` we use this because it can help reduce overfitting and lower the probability that the model will be overconfident in its predictions.

2. **Learning rate, scheduler, batch size, epochs, weight decay.**

   We start training with a learning rate of 1×10^{-4}, CosineAnnealingLR with a minLR of 1×10^{-5}.

3. **Hardware/software environment (GPU, PyTorch version).**

   We use a Tesla-V100-SXM2-32GB running with PyTorch:2.5.0a0+b465a5843b.nv24.9

4 Experiments

1. **Baseline setup.**

   All details about the setup are listed in the training and model sections.

2. **Hyperparameter tuning plan and validation method.**

   We do not employ any hyperparameter tuning for this project. The model setup is strong enough as is and I fear that hyperparameter tuning may lead to a biased result.

3. **Ablations**

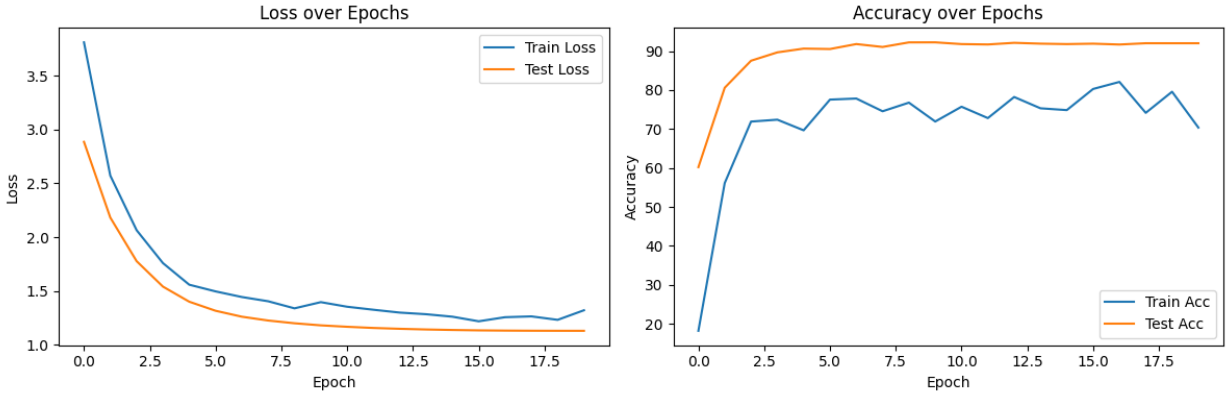
   We run a small a small set of ablations, results are in the table below.

Batch Size	LR	Routing	Accuracy
64	1E-4	✓	93.0983
64	1E-4	✘	93.4125
64	1E-5	✓	93.0983
64	1E-5	✘	93.4125
128	1E-4	✓	**93.6446**
128	1E-4	✘	92.5737
128	1E-5	✓	**93.6446**
128	1E-5	✘	92.4613

These results are averaged over all 10 folds validation sets. From these we can see that both the learning rate, batch size, and routing vs non-routing actually have a small effect on model performance.

5 Results

1. Training/validation accuracy (and loss) and any key curves/plots.



2. Kaggle public leaderboard score (and submission settings).

2 remvan 0.95454 14 1d

The averaged train/test loss and accuracy over the training epochs averaged over all K=10 folds of training.

6 Discussion

1. **What worked best and why.**
2. **Failure cases, overfitting/underfitting observations.**

   Initial experiments were with ConvNext models as an initial baseline, we noticed overfitting on these models, and poor learning. Likely because the dataset they were pretrained on is ImageNet whereas the SigLIP model is pretrained on the Webli dataset which consists of 100 billion image text pairs.

3. **Limitations and concrete next improvements.**

   One could further improve our implementation by better tuning the augmentations, running a hyperparameter search, experimenting with different architectures (unlikely to find much improvement here we tried DinoV3), ensembling multiple networks together.

7 Reproducibility

1. **Random seeds and determinism settings.** `seed=1666`. We set this for PyTorch, Numpy, and random.
2. **Package versions / environment setup steps.** We use the image nvcr.io/nvidia/pytorch:24.09-py3 and install the timm library. If you do this everything will run correctly.
3. **Exact commands to train and to generate submission.csv.**

   To train the submitted model run

```
python3 main.py \
    --backbone vit_so400m_patch14_siglip_378.v2_webli \
    --use_routing \
    --image_size 378 \
    --use_mixup \
    --epochs 20 \
    --seed 1666 \
    --batch_size 64
```

   To get the submitted test results run

```
python3 gather_test_preds.py \
    --output res_router.csv \
    --path Experiments/1666/vit_so400m_patch14_siglip_378.
       v2_webli_batch_size:64_img_size:378_mixup_routing
```

8 Team Contributions

1. **Evan Kaiden:** Training logic, Code for model setup and router network, test prediction gathering script
2. **Rami Al Habash:** Data setup, K-fold loop, metric visualization, data augmentations

9 References

1. **TorchVision documentation / pretrained model sources.**

   We use timm to load in our model via: `timm.create_model("vit_so400m_patch14_siglip_378.v2_webli", pretrained=True, num_classes=0)` the model card can be found here `https://huggingface.co/timm/vit_so400m_patch14_siglip_378.v2_webli`

2. **Any papers or tutorials used.** We used the following to

   - `https://albumentations.ai/docs/4-advanced-guides/test-time-augmentation/#tta-for-image-classification` — test time augmentation (tta)
   - `https://arxiv.org/pdf/2303.15343` — siglip
   - `https://arxiv.org/pdf/1905.04899` — cutmix
   - `https://arxiv.org/pdf/1710.09412` — mixup